Text Categorization

1. Import Required Libraries

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
```

Purpose

TfidfVectorizer

Converts text into numerical features using TF-IDF.

MultinomialNB

Naive Bayes algorithm commonly used for text classification.

Pipeline

Combines preprocessing and classification into a single workflow.

> Note: CountVectorizer is imported but not used in this program.

2. Training Documents

```
documents = [
    "machine learning improves prediction",
    "deep learning uses neural networks",
    "football is a popular sport",
    "cricket is played worldwide",
    "artificial intelligence is powerful",
    "soccer is an exciting game"
]
```

Dataset

The dataset contains six documents.

Technology Documents

```
machine learning improves prediction
deep learning uses neural networks
artificial intelligence is powerful
```

Sports Documents

```
football is a popular sport
cricket is played worldwide
soccer is an exciting game
```

These examples are used to train the classifier.

3. Class Labels

```
labels = [
    "Technology",
    "Technology",
    "Sports",
    "Sports",
    "Technology",
    "Sports"
]
```

Each document has a corresponding category.

Document	Category
machine learning improves prediction	Technology
deep learning uses neural networks	Technology
football is a popular sport	Sports
cricket is played worldwide	Sports
artificial intelligence is powerful	Technology
soccer is an exciting game	Sports

4. Build the Machine Learning Pipeline

```
model = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('classifier', MultinomialNB())
])
```

What Happens?

The pipeline performs two steps:

```
Raw Text
    ↓
TF-IDF Vectorization
```

```
    ↓
Naive Bayes Classification
    ↓
Predicted Category
```

5. TF-IDF Feature Extraction

```
TfidfVectorizer()
```

Why?

Machine learning algorithms cannot understand raw text directly.

Example:

```
deep learning uses neural networks
```

must be converted into numbers.

TF (Term Frequency)

Measures how often a word appears in a document.

Formula:

IDF (Inverse Document Frequency)

Measures how unique a word is.

Formula:

Where:

- N = total documents
- DF(t) = documents containing term t

TF-IDF

Words such as:

```
neural
network
intelligence
```

receive higher importance because they are more informative.

6. Naive Bayes Classifier

```
MultinomialNB()
```

Purpose

Learns word-category relationships from training data.

It calculates:

using Bayes' Theorem:

Where:

- C = category
- D = document

The category with the highest probability is selected.

7. Train the Model

```
model.fit(documents, labels)
```

What Happens?

The model learns:

Technology-related Words

```
machine
learning
deep
neural
networks
artificial
intelligence
prediction
```

Sports-related Words

```
football
cricket
soccer
sport
```

```
game
played
```

This training phase builds the classification model.

8. Test Document

```
test_text = [
    "neural network and artificial intelligence"
]
```

The sentence contains words like:

```
neural
network
artificial
intelligence
```

These words frequently appear in Technology examples.

9. Predict Category

```
prediction = model.predict(test_text)
```

The pipeline automatically:

Step 1

Converts text into TF-IDF features.

```
Text
↓
Numerical Vector
```

Step 2

Applies Naive Bayes.

```
Vector
↓
Probability Calculation
↓
Category Prediction
```

10. Display Result

```
print("Predicted Category:", prediction[0])
```

Expected Output:

```
Predicted Category: Technology
```

because the test sentence contains strongly technology-related terms.

Example Classification Process

Input

```
neural network and artificial intelligence
```

Important Words

```
neural
network
artificial
intelligence
```

Learned Association

```
Technology → High Probability
Sports     → Low Probability
```

Final Prediction

```
Technology
```

Real-World Applications

Text categorization is widely used in:

- Email Spam Detection
- News Classification
- Sentiment Analysis
- Document Organization

- Chatbots
- Customer Support Ticket Routing
- Social Media Monitoring

Program Workflow

```
Training Documents
        ↓
TF-IDF Feature Extraction
        ↓
Naive Bayes Training
        ↓
Learn Word Patterns
        ↓
Input New Text
         ↓
 TF-IDF Conversion
         ↓
 Category Prediction
```

Key NLP Concepts Used

Concept	Role
Text Classification	Assign text to categories
TF-IDF	Converts text into weighted numerical features
Naive Bayes	Classification algorithm
Pipeline	Automates preprocessing + prediction
Machine Learning	Learns patterns from labeled examples

Sample Interview Questions

Q1. Why is TF-IDF used?

Answer: TF-IDF converts text into numeric vectors while reducing the impact of common words and emphasizing important words.

Q2. Why is Naive Bayes popular for text classification?

Answer: It is simple, fast, memory-efficient, and performs well on high-dimensional text data.

Q3. What category will the test sentence belong to?

Answer: Technology, because it contains words like *neural*, *network*, and *artificial intelligence*.

Time Complexity

Let:

- **N** = number of documents
- **V** = vocabulary size

Training

```
TF-IDF Creation : O(N × V)
Naive Bayes Training : O(N × V)
```

Prediction

```
O(V)
```

Overall, this is an efficient approach for small and medium-sized text classification problems.

Key Takeaway

This program builds a **Text Categorization System** that uses **TF-IDF** to transform text into numerical features and **Naive Bayes** to classify the text into predefined categories such as **Technology** and **Sports**. This combination is one of the most common classical NLP approaches for document classification.